

Attention Is All You Need

Thursday, January 22, 2026 8:54 PM

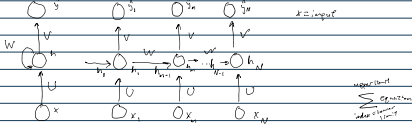
Recurrent Neural Network

These encode history up to current time t
Designed to apply the same type of operation
At each time, it's an instant via using the current state with the input value

Equations:

$$h_t = f(Ux_t + Wh_{t-1} + b)$$

U = weight
 W = weight
 b = bias
 x_t = input
 h_{t-1} = current state at time $t-1$
 b = output bias



Back Propagation

Applying gradient functions on N parameters with $N-1$ parameter outputs
states as input
Backwards of forward propagations
Equations:

$$Cost = J(U, W, V, b, c) = \sum_{n=1}^N J_n(U, W, V, b, c)$$

y = Ground truths

$$h_t = g(Vh_{t-1} + c)$$

g = sigmoid
 c = bias

$$\frac{\partial J}{\partial h_t} = \left(\frac{\partial J}{\partial h_{t+1}} \right) \left(\frac{\partial h_{t+1}}{\partial h_t} \right) + \left(\frac{\partial J}{\partial h_t} \right) \left(\frac{\partial h_t}{\partial h_t} \right)$$

T = time (latest time step)
 $h_t = \tanh(z_t)$

These equations are used for forward/backward passes
except: $h_{t-1}, h_{t-2}, \dots, h_{t-N}$

Basically, instead of counting up, you count down

Finding things:

$$\frac{\partial J}{\partial h_t} = \left(\frac{\partial J}{\partial h_{t+1}} \right) \left(\frac{\partial h_{t+1}}{\partial h_t} \right) + \left(\frac{\partial J}{\partial h_t} \right) \left(\frac{\partial h_t}{\partial h_t} \right)$$

$W(x)$ = the vector turned to a scalar!!
Causes Vanishing or explosive Gradients

- Deep RNNs

- Multiple layers

- Bidirectional RNNs

- both h_t and h_{t-1} state variables (forward and backward)

- sees forward and backward for recursion instead of just forward

Long Short Term Memory:

designed to help against vanishing gradient

Practice Equation

$$Input: \begin{bmatrix} 0.25 \\ 0.75 \\ 0.65 \end{bmatrix}$$

$$U = \begin{bmatrix} 0.25 & 0.15 \\ 0.33 & 0.46 \\ 0.46 & 0.82 \end{bmatrix} \quad W = \begin{bmatrix} 0.65 & 0.31 \\ 0.31 & 0.22 \end{bmatrix} \quad b = \begin{bmatrix} 0.12 & 0.61 & 0.76 \end{bmatrix}$$

x_t = input at time

h_{t-1} = previous state

U = hidden weight

W = output weight

b = bias

(technically, not output weight, but you can imagine it as the most external weight)

For standard RNN:

$$h_t = \tanh(z_t)$$

$$z_t = (Wx_t + Uh_{t-1}) + b$$

Given:

Hidden neurons = 2

Output classes = 2

Current time Previous time

Causes Exploding Gradients

$$z_t = (W \cdot x_t) + (U \cdot h_{t-1}) + b$$

$$h_t = \tanh(z_t)$$

$$\tanh = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\text{Matrix Tanh} = \sum_{i=0}^N \sum_{j=0}^M \frac{e^{x_{ij}} - e^{-x_{ij}}}{e^{x_{ij}} + e^{-x_{ij}}}$$

Forward with $N \times M$ dimensions

$$h_t = h_t$$

You can do this RECURSIVELY for the amount of hidden layers

Back Propagation Through Time:

Loss is calculated over time, so we have losses for each state

How to get loss from state?

Expected output if I assume

$$\tanh(z_t) = \tanh(z_{t-1}) \cdot (1 - b_t^2)$$

We need the gradients for all matrices

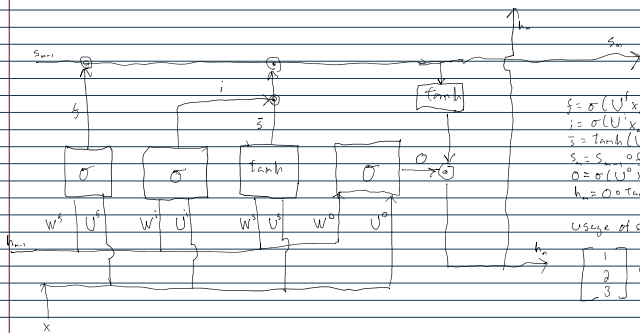
$$\frac{\partial J}{\partial h_t} = \frac{\partial J}{\partial z_t} \cdot \frac{\partial z_t}{\partial h_t}$$

$\frac{\partial J}{\partial h_t}$ = Loss function?

$$\frac{\partial J}{\partial h_t} = \frac{\partial J}{\partial z_t} \cdot \frac{\partial z_t}{\partial h_t}$$

MSE = Loss =

$$L_{MSE} = \sum_{n=0}^N \frac{1}{2} (y_{in} - y_{out})^2$$



$f = \sigma(U_f x_t + W_f h_{t-1} + b_f)$ need forget weights
 $i = \sigma(U_i x_t + W_i h_{t-1} + b_i)$ need input gate
 $\tilde{c} = \tanh(U_{\tilde{c}} x_t + W_{\tilde{c}} h_{t-1} + b_{\tilde{c}})$ need candidate memory
 $o = \sigma(U_o x_t + W_o h_{t-1} + b_o)$ need output gate
 $h_t = o \cdot \tanh(\tilde{c})$ need output and candidate memory

Usage of σ :

$$\begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \cdot \begin{bmatrix} 3 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \cdot 3 \\ 2 \cdot 0 \\ 3 \cdot 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 0 \\ 3 \end{bmatrix}$$